



PROJECT ALGORITMA DAN PEMROGRAMAN



DOSEN PENGAMPU : LUTFIAH MAHARANI SINIWI S.STAT, M.STAT.

ANGGOTA KELOMPOK F :

- 1.Elhan Della Yolanda (K1D02590)
- 2.Fahridzal Fahmi Asidiq (K1D025100)
- 3.Sarah Alimunnisa (K1D025104)
- 4.Fauzan Taufik Ismail (K1D025108)



LATAR BELAKANG

Algoritma dan pemrograman merupakan dasar dalam pengembangan perangkat lunak dan penyelesaian berbagai permasalahan komputasional. Kemampuan menyusun algoritma yang logis serta mengimplementasikannya ke dalam bahasa pemrograman menjadi keterampilan penting bagi mahasiswa di bidang teknologi dan sains.

Melalui project ini, mahasiswa menerapkan konsep algoritma dan pemrograman untuk menyelesaikan enam studi kasus yang berbeda menggunakan bahasa Python dan R. Kegiatan ini bertujuan untuk melatih kemampuan analisis masalah, perancangan algoritma, implementasi program, serta pengujian hasil sehingga diperoleh solusi yang tepat dan sistematis.

[NEXT SLIDE](#)

TUJUAN PROJECT

- Memahami dan menerapkan konsep dasar algoritma dan pemrograman dalam menyelesaikan berbagai permasalahan komputasional.
- Menyusun solusi masalah dalam bentuk analisis, pseudocode, dan flowchart secara sistematis.
- Mengimplementasikan algoritma menggunakan bahasa pemrograman Python dan R.
- Melatih kemampuan menggunakan variabel, percabangan, fungsi, string, dan operasi matematika dalam program.
- Menguji dan mengevaluasi program melalui berbagai skenario pengujian untuk memastikan hasil yang benar.
- Mengembangkan kemampuan kerja sama tim, dokumentasi program, dan presentasi hasil project.



NEXT SLIDE

PROGRAM 1

MENGHITUNG JUMLAH KATA DAN JUMLAH KALIMAT

Program dibuat untuk menghitung jumlah kata dan jumlah kalimat dari sebuah teks. Sesuai ketentuan soal, diasumsikan bahwa teks tidak memuat tanda titik selain sebagai penanda akhir kalimat.

Program menerima sebuah teks sebagai input, kemudian:

- Menghitung jumlah kata berdasarkan jumlah spasi pada teks.
- Menghitung jumlah kalimat berdasarkan jumlah tanda titik (.).

Hasil akhir yang ditampilkan adalah jumlah kalimat dan jumlah kata yang terdapat dalam teks.

Kondisi	Hasil
Teks terdiri dari satu kalimat	Jumlah kalimat = 1
Teks terdiri dari beberapa kalimat	Jumlah kalimat sesuai jumlah titik
Teks memiliki banyak kata	Jumlah kata dihitung berdasarkan jumlah spasi + 1



PSUEDECODE

START

DEKLARASI

TEKS : STRING
JUMLAH_KATA : INTEGER
JUMLAH_KALIMAT : INTEGER

INPUT

TEKS

PROSES

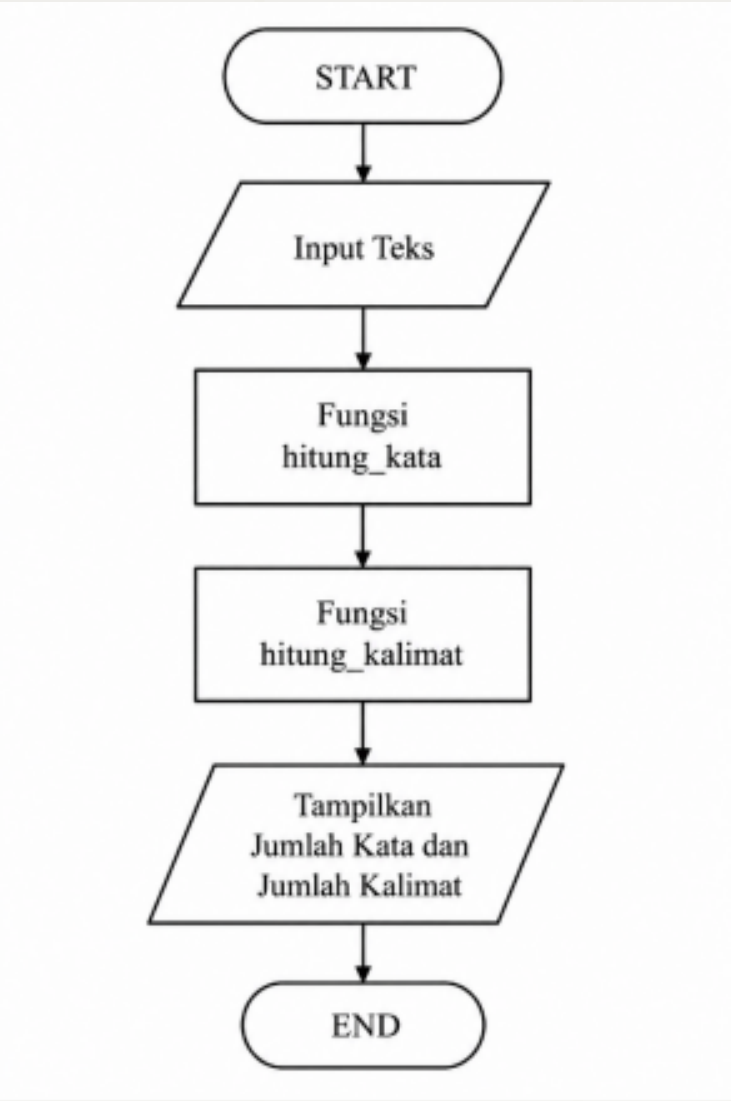
BUAT FUNGSI HITUNG_KATA(TEKS)
BUAT FUNGSI HITUNG_KALIMAT(TEKS)
JUMLAH_KATA ← HITUNG_KATA(TEKS)
JUMLAH_KALIMAT ← HITUNG_KALIMAT(TEKS)

OUTPUT

TAMPILKAN JUMLAH_KALIMAT
TAMPILKAN JUMLAH_KATA

END

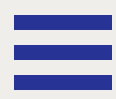
FLOWCHART



CODE PYTHON

CODE R

TABEL UJI



```
# PROGRAM 1
# MENGHITUNG JUMLAH KATA DAN JUMLAH KALIMAT

# MENGHITUNG JUMLAH KATA

def hitung_kata(teks):

    jumlah = 1

    for huruf in teks:

        if huruf == " ":

            jumlah = jumlah + 1

    return jumlah

# MENGHITUNG JUMLAH KALIMAT

def hitung_kalimat(teks):

    jumlah = 0

    for huruf in teks:

        if huruf == ".":

            jumlah = jumlah + 1

    return jumlah
```

```
# PROGRAM 1
# MENGHITUNG JUMLAH KATA DAN JUMLAH KALIMAT

# MENGHITUNG JUMLAH KATA

hitung_kata <- function(teks){

    jumlah <- 1

    for(huruf in strsplit(teks, "")[[1]]){

        if(huruf == " "){

            jumlah <- jumlah + 1

        }

    }

    return(jumlah)

}

# MENGHITUNG JUMLAH KALIMAT

hitung_kalimat <- function(teks){

    jumlah <- 0

    for(huruf in strsplit(teks, "")[[1]]){

        if(huruf == "."){

            jumlah <- jumlah + 1

        }

    }

    return(jumlah)

}
```

Data Uji (Input)	Jumlah Kalimat yang Diharapkan	Jumlah Kata yang Diharapkan	Hasil Program	Status
Membaca buku merupakan kegiatan	4	36	4 kalimat, 36 kata	Berhasil
Membaca buku merupakan kegiatan	2	14	2 kalimat, 14 kata	Berhasil
Membaca buku merupakan kegiatan	1	7	1 kalimat, 7 kata	Berhasil
Statistika.	1	1	1 kalimat, 1 kata	Berhasil
A. B. C. D. E.	5	5	5 kalimat, 5 kata	Berhasil

PROGRAM 2

MEMBUAT OBJEK STRING K1 SAMPAI DENGAN K4

Program ini digunakan untuk membuat dan menampilkan beberapa objek string yang mengandung tanda petik dan karakter khusus. Pengguna diminta membuat empat string, yaitu K1, K2, K3, dan K4, agar menghasilkan output yang sesuai dengan ketentuan soal. Program ini bertujuan untuk memahami cara penulisan string yang mengandung karakter khusus, seperti tanda petik tunggal dan ganda, dengan menggunakan teknik yang tepat, seperti escape character, dalam bahasa pemrograman Python dan R.

Kondisi	Hasil
String mengandung tanda petik tunggal	Ditampilkan sesuai format yang diminta
String mengandung tanda petik ganda	Ditampilkan sesuai format yang diminta
String mengandung kombinasi tanda petik Tunggal dan ganda	Ditampilkan tanpa error
String dicetak ke layar	Output sama persis dengan contoh soal

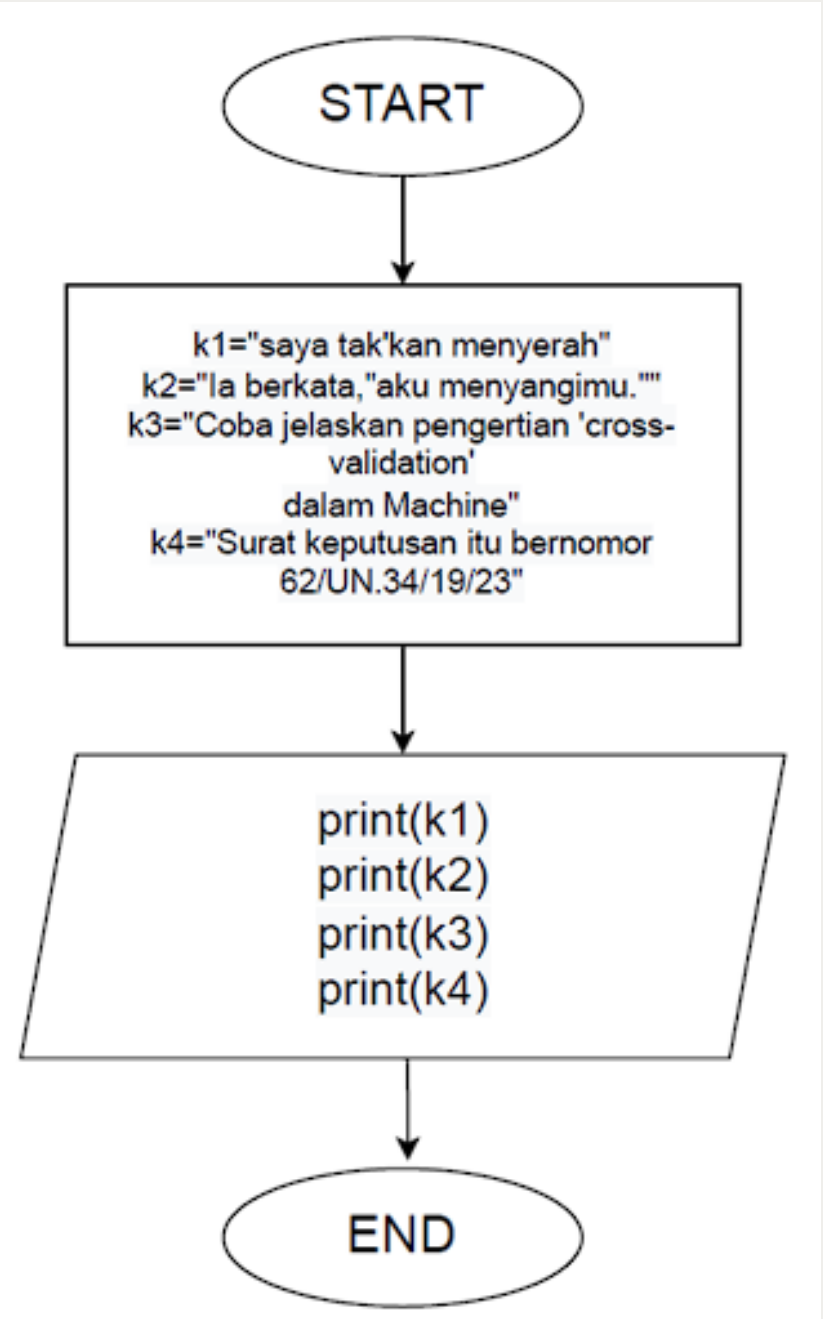
START

DEKLARASI
K1, K2, K3, K4 : string

PROSES
K1 ← "Saya tak 'kan menyerah."
K2 ← 'Ia berkata, "Aku menyangimu."'"
K3 ← "'Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
K4 ← "Surat ayerle itu bernomor 62/UN.34/19/2023."

OUTPUT
Tampilkan K1
Tampilkan K2
Tampilkan K3
Tampilkan K4

END



KODE PYTHON

```
# PROGRAM 2
# Membuat objek string K1 sampai K4

K1 = "Saya tak 'kan menyerah."

K2 = 'Ia berkata, "Aku menyayangimu."'

K3 = "Coba jelaskan pengertian 'cross-validation'
dalam Machine Learning!"

K4 = "Surat keputusan itu bernomor
62/UN.34/19/2023."

print(K1)
print(K2)
print(K3)
print(K4)
```

KODE R

```
# PROGRAM 2
# Membuat objek string K1 sampai K4

K1 <- "Saya tak 'kan menyerah."

K2 <- 'Ia berkata, "Aku menyayangimu."'

K3 <- "Coba jelaskan pengertian 'cross-validation'
dalam Machine Learning!"

K4 <- "Surat keputusan itu bernomor
62/UN.34/19/2023."

cat (k1, "\n")
cat (k2, "\n")
cat (k3, "\n")
cat (k4, "\n")
```



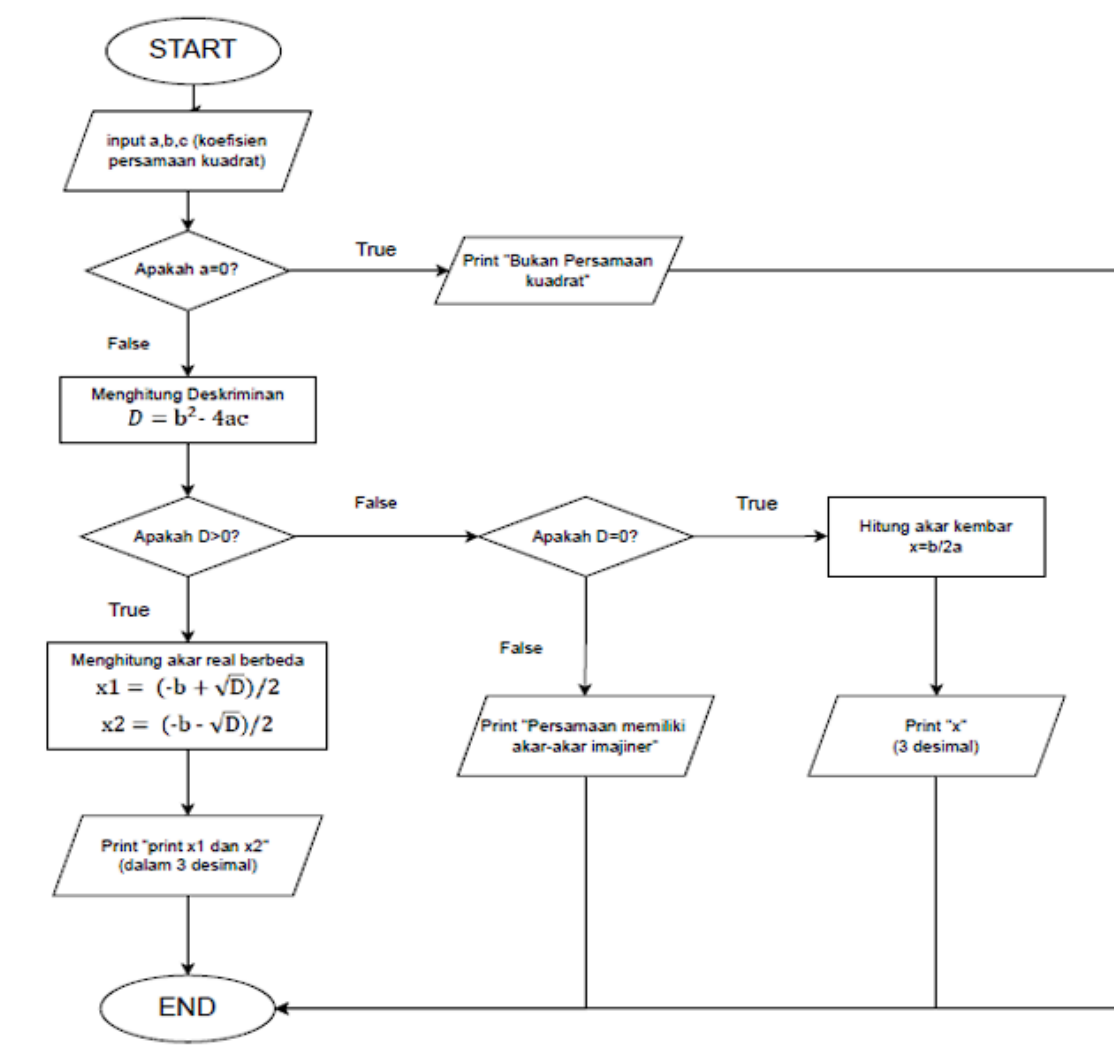
PROGRAM 3

MENGHITUNG NILAI X DARI PERSAMAAN KUADRAT



Program ini digunakan untuk menentukan akar-akar persamaan kuadrat $ax^2 + bx + c = 0$. Pengguna memasukkan koefisien-koefisien a , b , dan c . Jika a bernilai 0 maka persamaan tersebut bukan persamaan kuadrat. Program kemudian menghitung nilai diskriminan ($b^2 - 4ac$) untuk menentukan jenis akar-akar persamaan tersebut. Jika diskriminan bernilai positif, persamaan memiliki dua akar real berbeda. Jika diskriminan bernilai nol, persamaan memiliki dua akar real kembar. Jika diskriminan bernilai negatif, persamaan hanya memiliki akar-akar imajiner. Akar real ditampilkan dalam tiga desimal.

```
START
DEKLARASI
a, b, c : real
D : real
x, x1, x2 : real
INPUT
Input a, b, c
PROSES
if a = 0 then
    print "Bukan persamaan kuadrat"
else then
    D ←  $b^2 - 4ac$ 
    if D > 0 then
         $x1 \leftarrow (-b + \sqrt{D}) / (2 \times a)$ 
         $x2 \leftarrow (-b - \sqrt{D}) / (2 \times a)$ 
        Tampilkan "Persamaan memiliki dua akar real berbeda"
        Tampilkan "x1 = ", x1 (3 desimal)
        Tampilkan "x2 = ", x2 (3 desimal)
    else then
        if D = 0 then
             $x \leftarrow -b / (2 \times a)$ 
            Tampilkan "Persamaan memiliki akar real kembar"
            Tampilkan "x = ", x (3 desimal)
        else
            Tampilkan "Persamaan memiliki akar-akar imajiner"
        end
    end
end
end
END
```



Kondisi	Hasil
$A = 0$	Tampilkan bahwa persamaan tersebut bukan persamaan kuadrat
$\text{Diskriminan} > 0$	Dua akar real berbeda ditampilkan
$\text{Diskriminan} = 0$	Dua akar real kembar ditampilkan
$\text{Diskriminan} < 0$	Tampilkan bahwa persamaan hanya memiliki akar-akar imajiner



```
#PROGRAM NOMOR 3
import math
#Input koefisien persamaan kuadrat
a, b, c= #menginput koefisien persamaan kuadrat
# Input koefisien persamaan kuadrat
a, b, c = 1, 2, 5
#cek apakah persamaan merupakan persamaan kuadrat
if a == 0:
    print("Bukan persamaan kuadrat")
else:
#Hitung Diskriminan
    diskriminan = b**2 - 4*a*c
#cek nilai deskriminan
    if diskriminan > 0:
        x1 = (-b + math.sqrt(diskriminan)) / (2*a)
        x2 = (-b - math.sqrt(diskriminan)) / (2*a)
        print(f"Persamaan memiliki dua akar real berbeda:")
        print(f"x1 = {x1:.3f}")
        print(f"x2 = {x2:.3f}")
    elif diskriminan == 0:
        x = -b / (2*a)
        print(f"Persamaan memiliki akar real kembar:")
        print(f"x1 = x2 = {x:.3f}")
    else:
        print("Persamaan hanya memiliki akar-akar imajiner.")
```

```
# PROGRAM 3
# Akar-akar Persamaan Kuadrat
# Input koefisien persamaan kuadrat a,b,c
a<- ; b<- ; c<- ; #menginput nilai a, b, c
#Mengecek nilai koefisien a
if (a == 0){
    print("Bukan persamaan kuadrat")
}else {
#Menghitung diskriminan
    diskriminan <- b^2 - 4*a*c
#mengecek nilai diskriminan
    if (diskriminan > 0) {
        x1 <- (-b + sqrt(diskriminan)) / (2*a)
        x2 <- (-b - sqrt(diskriminan)) / (2*a)
        cat("Persamaan memiliki dua akar real berbeda:\n")
        cat(sprintf("x1 = %.3f\n", x1))
        cat(sprintf("x2 = %.3f\n", x2))
    } else if (diskriminan == 0) {
        x <- -b / (2*a)
        cat("Persamaan memiliki akar real kembar:\n")
        cat(sprintf("x1 = x2 = %.3f\n", x))
    } else {
        cat("Persamaan hanya memiliki akar-akar imajiner.\n")
    }
}
```

No	Input (a,b,c)	Kondisi D	Output yang Diharpakn	Status
1	a=1, b=-5, c=6	D=1 >0	x1= 3.000, x2=2.000	Sesuai
2	a=1, b=-4, c=4	D=0	x1=x2=2.000	Sesuai
3	a=1, b=2, c=5	D=-56 <0	Akar imajiner	Sesuai

PROGRAM 4

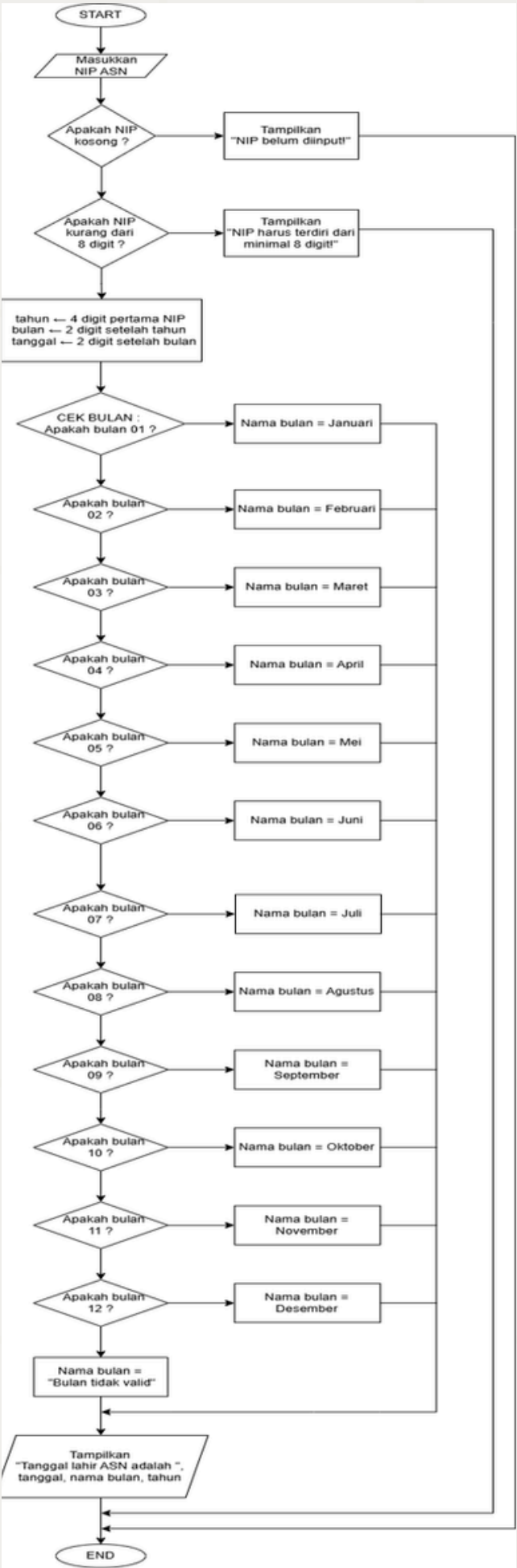
MENAMPILKAN TANGGAL LAHIR ASN BERDASARKAN NIP

Setiap ASN memiliki Nomor Induk Pegawai (NIP) yang memuat informasi tanggal lahir pada 8 digit pertama. Buatlah program yang menerima input NIP, kemudian menampilkan tanggal lahir dalam format DD NamaBulan YYYY. Gunakan struktur IF-ELSE IF-ELSE untuk menentukan nama bulan. Program juga harus menangani input tidak valid, seperti NIP terlalu pendek, tidak menginput NIP, atau bulan di luar rentang 01-12. Implementasikan program dalam Python dan R.

Kondisi	Hasil
NIP terdiri dari 18 digit dan bulan valid (01-12)	Tanggal lahir ASN berhasil ditampilkan
NIP valid dengan panjang tepat 8 digit	Tanggal lahir tetap berhasil ditampilkan (hanya 8 digit pertama yang digunakan)
NIP kurang dari 8 digit	Tanggal lahir tidak dapat ditentukan, dan tampilkan pesan : “NIP harus terdiri dari minimal 8 digit!”
NIP kosong (tidak diisi)	Program tidak dapat menentukan tanggal lahir, dan tampilkan pesan : “NIP belum diinput!”
Bulan di luar 01-12	Tampilkan pesan: “Bulan tidak valid”

```
START
DEKLARASI
  nip : STRING
  tahun, bulan, tanggal : STRING
  nama_bulan : STRING
INPUT
  NIP
PROSES
  IF nip == "" THEN
    Tampilkan "NIP belum diinput!"
  ELSE IF LENGTH(nip) < 8 THEN
    Tampilkan "NIP harus terdiri dari minimal 8 digit!"
  ELSE
    tahun ← 4 digit pertama NIP
    bulan ← 2 digit setelah tahun
    tanggal ← 2 digit setelah bulan

    IF bulan = "01" THEN
      namaBulan ← "Januari"
    ELSE IF bulan = "02" THEN
      namaBulan ← "Februari"
    ELSE IF bulan = "03" THEN
      namaBulan ← "Maret"
    ELSE IF bulan = "04" THEN
      namaBulan ← "April"
    ELSE IF bulan = "05" THEN
      namaBulan ← "Mei"
    ELSE IF bulan = "06" THEN
      namaBulan ← "Juni"
    ELSE IF bulan = "07" THEN
      namaBulan ← "Juli"
    ELSE IF bulan = "08" THEN
      namaBulan ← "Agustus"
    ELSE IF bulan = "09" THEN
      namaBulan ← "September"
    ELSE IF bulan = "10" THEN
      namaBulan ← "Oktober"
    ELSE IF bulan = "11" THEN
      namaBulan ← "November"
    ELSE IF bulan = "12" THEN
      namaBulan ← "Desember"
    ELSE
      nama_bulan ← "Bulan tidak valid"
    END IF
  END IF
OUTPUT
  "Tanggal lahir ASN adalah ", tanggal, nama_bulan, tahun
END
```





```
nip = "199301212019031010"

if nip == "":
    print("NIP belum diinput!")
elif len(nip) < 8:
    print("NIP harus terdiri dari minimal 8 digit!")
else:
    tahun = nip[0:4]
    bulan = nip[4:6]
    tanggal = nip[6:8]

    if bulan == "01":
        nama_bulan = "Januari"
    elif bulan == "02":
        nama_bulan = "Februari"
    elif bulan == "03":
        nama_bulan = "Maret"
    elif bulan == "04":
        nama_bulan = "April"
    elif bulan == "05":
        nama_bulan = "Mei"
    elif bulan == "06":
        nama_bulan = "Juni"
    elif bulan == "07":
        nama_bulan = "Juli"
    elif bulan == "08":
        nama_bulan = "Agustus"
    elif bulan == "09":
        nama_bulan = "September"
    elif bulan == "10":
        nama_bulan = "Oktober"
    elif bulan == "11":
        nama_bulan = "November"
    elif bulan == "12":
        nama_bulan = "Desember"
    else:
        nama_bulan = "Bulan tidak valid"

print("Tanggal lahir ASN adalah", int(tanggal), nama_bulan, tahun)
```

```
nip <- "199301212019031010"

if (nip == "") {
  print("NIP belum diinput!")
} else if (nchar(nip) < 8) {
  print("NIP kurang dari 8 digit, tanggal lahir tidak dapat ditentukan!")
} else {
  tahun <- substr(nip, 1, 4)
  bulan <- substr(nip, 5, 6)
  tanggal <- substr(nip, 7, 8)

  if (bulan == "01") {
    nama_bulan <- "Januari"
  } else if (bulan == "02") {
    nama_bulan <- "Februari"
  } else if (bulan == "03") {
    nama_bulan <- "Maret"
  } else if (bulan == "04") {
    nama_bulan <- "April"
  } else if (bulan == "05") {
    nama_bulan <- "Mei"
  } else if (bulan == "06") {
    nama_bulan <- "Juni"
  } else if (bulan == "07") {
    nama_bulan <- "Juli"
  } else if (bulan == "08") {
    nama_bulan <- "Agustus"
  } else if (bulan == "09") {
    nama_bulan <- "September"
  } else if (bulan == "10") {
    nama_bulan <- "Oktober"
  } else if (bulan == "11") {
    nama_bulan <- "November"
  } else if (bulan == "12") {
    nama_bulan <- "Desember"
  } else {
    nama_bulan <- "Bulan tidak valid"
  }

  print(paste("Tanggal lahir ASN adalah", as.numeric(tanggal),
nama_bulan, tahun))
}
```

<u>Skenario</u>	Input	Output yang diharapkan	Output	Status
<u>Skenario 1 (Normal)</u>	“199301212019031010”	<u>Tanggal lahir ASN adalah 21 Januari 1993</u>	<u>Tanggal lahir ASN adalah 21 Januari 1993</u>	<u>Berhasil</u>
<u>Skenario 2 (Kondisi Khusus)</u>	“200613192019031010”	<u>Tanggal lahir ASN adalah 19 Bulan tidak valid 2006</u>	<u>Tanggal lahir ASN adalah 19 Bulan tidak valid 2006</u>	<u>Berhasil</u>
<u>Skenario 3 (Kondisi Khusus)</u>	“200705”	<u>NIP harus terdiri dari minimal 8 digit!</u>	<u>NIP harus terdiri dari minimal 8 digit!</u>	<u>Berhasil</u>
<u>Skenario 4 (Kondisi Khusus)</u>	“”	<u>NIP belum diinput!</u>	<u>NIP belum diinput!</u>	<u>Berhasil</u>



PROGRAM 5

PROGRAM UNTUK MENENTUKAN APAKAH TITIK TERSEBUT TERGOLONG CLUSTER A, B, ATAU C

PROGRAM INI DIGUNAKAN UNTUK MENENTUKAN CLUSTER TERDEKAT DARI SUATU TITIK DATA U BERDASARKAN METODE JARAK EUCLIDEAN. DIKETAHUI TIGA PUSAT CLUSTER:
CLUSTER A = (2, 1, 3)
CLUSTER B = (1, -4, 6)
CLUSTER C = (-2, 3, -2)
PROGRAM AKAN MENGHITUNG JARAK TITIK U TERHADAP MASING-MASING PUSAT CLUSTER MENGGUNAKAN RUMUS EUCLIDEAN. SETELAH DIPEROLEH KETIGA JARAK TERSEBUT, PROGRAM AKAN MEMBANDINGKAN NILAI JARAK DAN MENENTUKAN CLUSTER YANG MEMILIKI JARAK PALING KECIL.

Kondisi	Hasil
Jarak A paling kecil	Cluster A
Jarak B paling kecil	Cluster B
Jarak C paling kecil	Cluster C
Terdapat jarak yang sama	Tidak Diketahui

PSTART

DEKLARASI

x1, x2, x3 : real
jarakA : real
jarakB : real
jarakC : real
cluster : string

INPUT

x1
x2
x3

PROSES

Buat fungsi hitung_jarak()

jarak ← sqrt(
 (x1-y1)^2 +
 (x2-y2)^2 +
 (x3-y3)^2
)

Buat fungsi hitung_semua_jarak()

jarakA ← hitung_jarak(x1,x2,x3,2,1,3)

jarakB ← hitung_jarak(x1,x2,x3,1,-4,6)

jarakC ← hitung_jarak(x1,x2,x3,-2,3,-2)

Jika jarakA < jarakB dan jarakA < jarakC

 cluster ← "A"

Jika tidak, jika jarakB < jarakA dan jarakB < jarakC

 cluster ← "B"

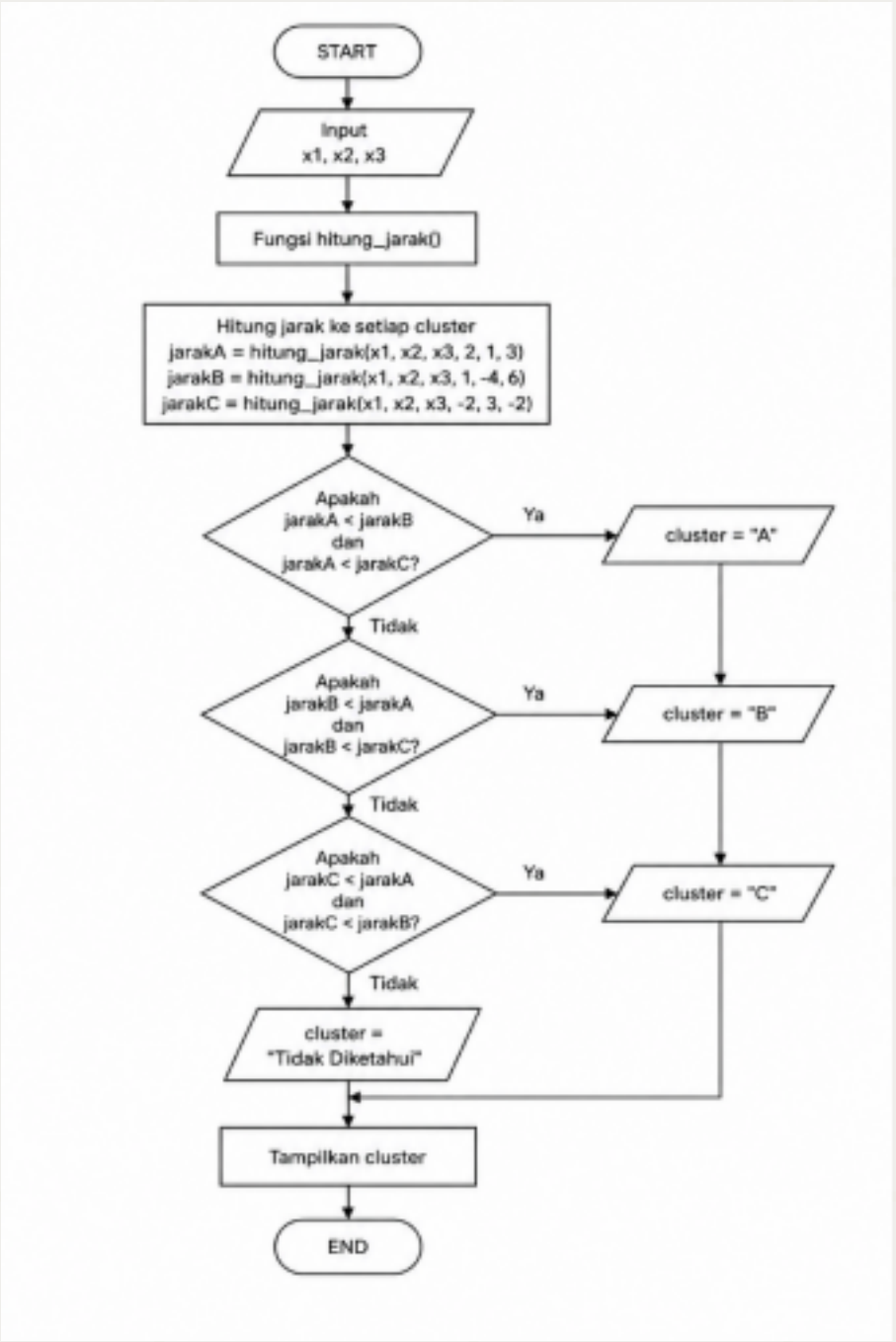
Jika tidak, jika jarakC < jarakA dan jarakC < jarakB

 cluster ← "C"

Selain itu

 cluster ← "Tidak Diketahui"

OUTPUT





KODE PYTHON

```
# PROGRAM 5
# Menentukan Cluster Terdekat

import math

# Fungsi Menghitung Jarak Euclidean

def hitung_jarak(x1, x2, x3, c1, c2, c3):

    return math.sqrt(
        (x1 - c1) ** 2 +
        (x2 - c2) ** 2 +
        (x3 - c3) ** 2

# Fungsi Menghitung Semua Jarak

def hitung_semua_jarak(x1, x2, x3):

    jarakA = hitung_jarak(x1, x2, x3, 2, 1, 3)

    jarakB = hitung_jarak(x1, x2, x3, 1, -4, 6)

    jarakC = hitung_jarak(x1, x2, x3, -2, 3, -2)

    return jarakA, jarakB, jarakC
```

KODE R

```
# PROGRAM 5
# Menentukan Cluster Terdekat

# Fungsi Menghitung Jarak Euclidean

hitung_jarak <- function(x1, x2, x3, c1, c2, c3){

    sqrt(
        (x1 - c1)^2 +
        (x2 - c2)^2 +
        (x3 - c3)^2
    )

}

# Fungsi Menghitung Semua Jarak

hitung_semua_jarak <- function(x1, x2, x3){

    jarakA <- hitung_jarak(x1, x2, x3, 2, 1, 3)

    jarakB <- hitung_jarak(x1, x2, x3, 1, -4, 6)

    jarakC <- hitung_jarak(x1, x2, x3, -2, 3, -2)

    return(list(jarakA, jarakB, jarakC))

}
```

TABEL UJI

Data Uji (Input)	Output yang Diharapkan	Hasil Program	Status
$x_1 = 2, x_2 = 1, x_3 = 3$	Titik U termasuk Cluster A	Titik U termasuk Cluster A	Berhasil
$x_1 = 1, x_2 = -4, x_3 = 6$	Titik U termasuk Cluster B	Titik U termasuk Cluster B	Berhasil
$x_1 = -2, x_2 = 3, x_3 = -2$	Titik U termasuk Cluster C	Titik U termasuk Cluster C	Berhasil



PROGRAM 6

MENERIMA MASUKAN PROPORSI SAMPEL $\hat{p}$ SERTA UKURAN KEPERCAYAAN α DANMENYAJIKAN NILAI INTERVAL KONFIDENSI YANG SESUAI.

Interval konfidensi $(1 - \alpha)$ untuk proporsi populasi p dapat dihitung sebagai

$$\hat{p} - z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} < p < \hat{p} + z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

dengan z bernilai 1,645 untuk α sebesar 10% dan 1,96 untuk α sebesar 5%.
Buatlah sebuah program yang menerima masukan proporsi sampel $\hat{p}$ serta ukuran kepercayaan α danmenyajikan nilai interval konfidensi yang sesuai.
Program harus menampilkan pesan kesalahan bila proporsi yang dimasukkan bernilai < 0 atau > 1 .

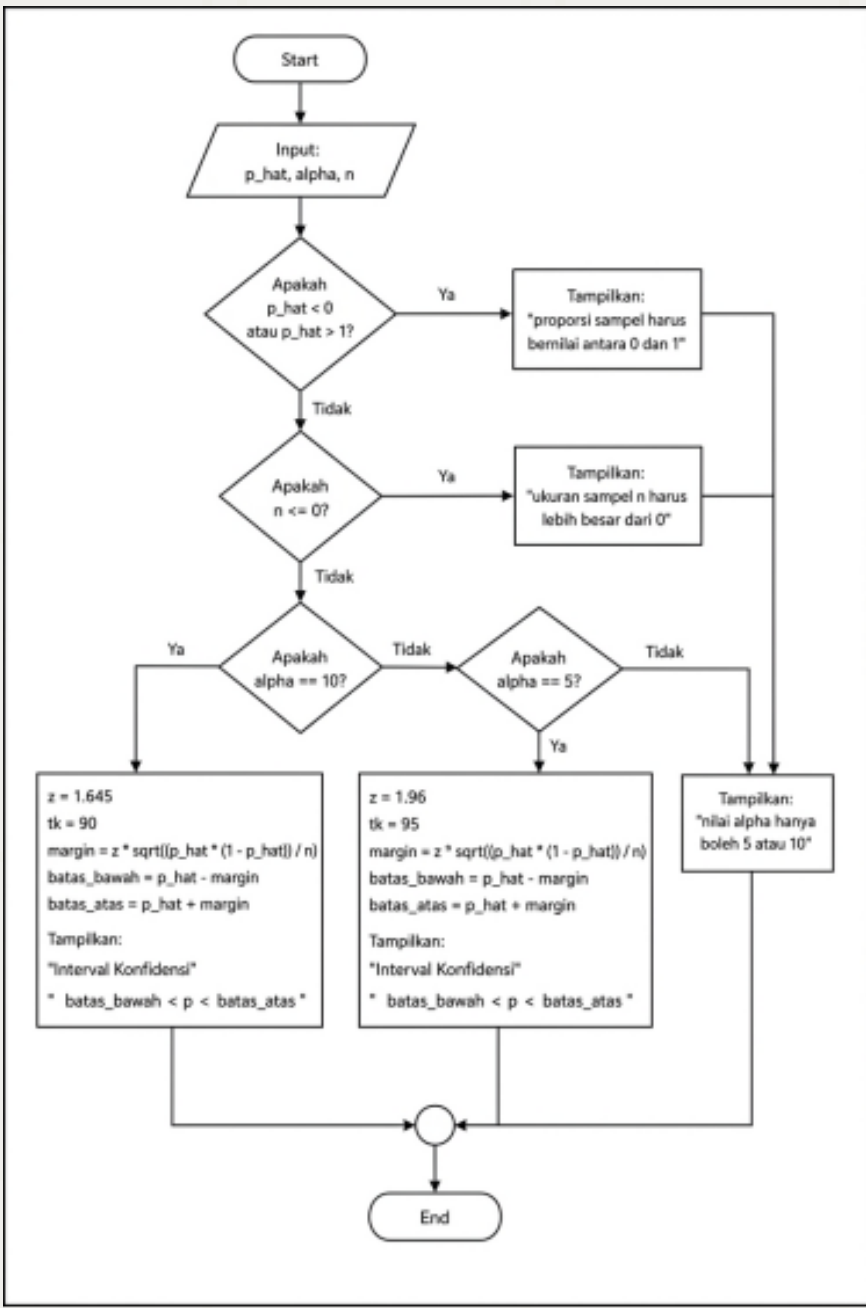
Kondisi	Hasil
$\hat{p} < 0$ atau $\hat{p} > 1$	Menampilkan pesan: "proporsi sampel harus bernilai antara 0 dan 1"
$n \leq 0$	Menampilkan pesan: " ukuran sampel n harus lebih besar dari 0"
$\alpha = 10$	Menggunakan $Z = 1.645$ untuk Tingkat Kepercayaan 90%
$\alpha = 5$	Menggunakan $Z = 1.96$ untuk Tingkat Kepercayaan 95%
α selain 5 atau 10	Menampilkan pesan: "nilai alpha hanya boleh 5 atau 10"

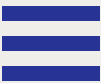
PSUDECODE

```
START
DEKLARASI
p_hat, z, margin, batas_bawah, batas_atas : real
alpha, n, tk : integer
INPUT
p_hat, alpha, n

IF p_hat < 0 ATAU p_hat > 1 THEN
TAMPILKAN "proporsi sampel harus bernilai antara 0 dan 1"
ELSE IF n <= 0 THEN
TAMPILKAN "Error: ukuran sampel n harus lebih besar dari 0"
ELSE IF alpha == 10 THEN
z ← 1.645
tk ← 90
margin ← z * SQRT((p_hat * (1 - p_hat)) / n)
batas_bawah ← p_hat - margin
batas_atas ← p_hat + margin
TAMPILKAN "Interval Konfidensi"
TAMPILKAN "batas_bawah < p < batas_atas"
ELSE IF alpha == 5 THEN
z ← 1.96
tk ← 95
margin ← z * SQRT((p_hat * (1 - p_hat)) / n)
batas_bawah ← p_hat - margin
batas_atas ← p_hat + margin
TAMPILKAN "Interval Konfidensi tk%"
TAMPILKAN "batas_bawah < p < batas_atas"
ELSE
TAMPILKAN "nilai alpha hanya boleh 5 atau 10"
END
```

FLOWCHART





CODE PYTHON

```
# PROGRAM 6
# Menghitung Interval Konfidensi Proporsi Sampel
import math

def hitung_interval_konfidensi(p_hat, alpha, n):
    if p_hat < 0 or p_hat > 1:
        print("proporsi sampel harus bernilai antara 0 dan 1")
    elif n <= 0:
        print("ukuran sampel n harus lebih besar dari 0")
    elif alpha == 10:
        z = 1.645
        tk = 90
        margin = z * math.sqrt((p_hat * (1 - p_hat)) / n)
        batas_bawah = p_hat - margin
        batas_atas = p_hat + margin
        print(f"Interval Konfidensi")
        print(f" {batas_bawah:.4f} < p < {batas_atas:.4f}")
    elif alpha == 5:
        z = 1.96
        tk = 95
        margin = z * math.sqrt((p_hat * (1 - p_hat)) / n)
        batas_bawah = p_hat - margin
        batas_atas = p_hat + margin
        print(f"Interval Konfidensi")
        print(f" {batas_bawah:.4f} < p < {batas_atas:.4f}")
    else:
        print("nilai alpha hanya boleh 5 atau 10")
```

CODE R

```
# PROGRAM 6
# Menghitung Interval Konfidensi Proporsi Sampel

hitung_interval_konfidensi <- function(p_hat, alpha, n) {
  if (p_hat < 0 || p_hat > 1) {
    cat("proporsi sampel harus bernilai antara 0 dan 1")
  } else if (n <= 0) {
    cat("ukuran sampel n harus lebih besar dari 0")
  } else if (alpha == 10) {
    z <- 1.645
    tk <- 90
    margin <- z * sqrt((p_hat * (1 - p_hat)) / n)
    batas_bawah <- p_hat - margin
    batas_atas <- p_hat + margin
    cat(sprintf("Interval Konfidensi"))
    cat(sprintf(" %.4f < p < %.4f", batas_bawah, batas_atas))
  } else if (alpha == 5) {
    z <- 1.96
    tk <- 95
    margin <- z * sqrt((p_hat * (1 - p_hat)) / n)
    batas_bawah <- p_hat - margin
    batas_atas <- p_hat + margin
    cat(sprintf("Interval Konfidensi"))
    cat(sprintf(" %.4f < p < %.4f", batas_bawah, batas_atas))
  } else {
    cat("nilai alpha hanya boleh 5 or 10\n")
  }
}
```

TABEL UJI

Data Uji (Input)	Output yang Diharapkan	Output Program	Status
$\hat{p} = 0.6, n = 100, \alpha = 5\%$	Interval Konfidensi $95\% 0.5040 < p < 0.6960$	Interval Konfidensi $95\% 0.5040 < p < 0.6960$	Berhasil
$\hat{p} = 0.6, n = 100, \alpha = 10\%$	Interval Konfidensi $90\% 0.5194 < p < 0.6806$	Interval Konfidensi $90\% 0.5194 < p < 0.6806$	Berhasil
$\hat{p} = 1.5, n = 100, \alpha = 5\%$	Pesan kesalahan: "proporsi sampel harus bernilai antara 0 dan 1"	Pesan kesalahan: "proporsi sampel harus bernilai antara 0 dan 1"	Berhasil